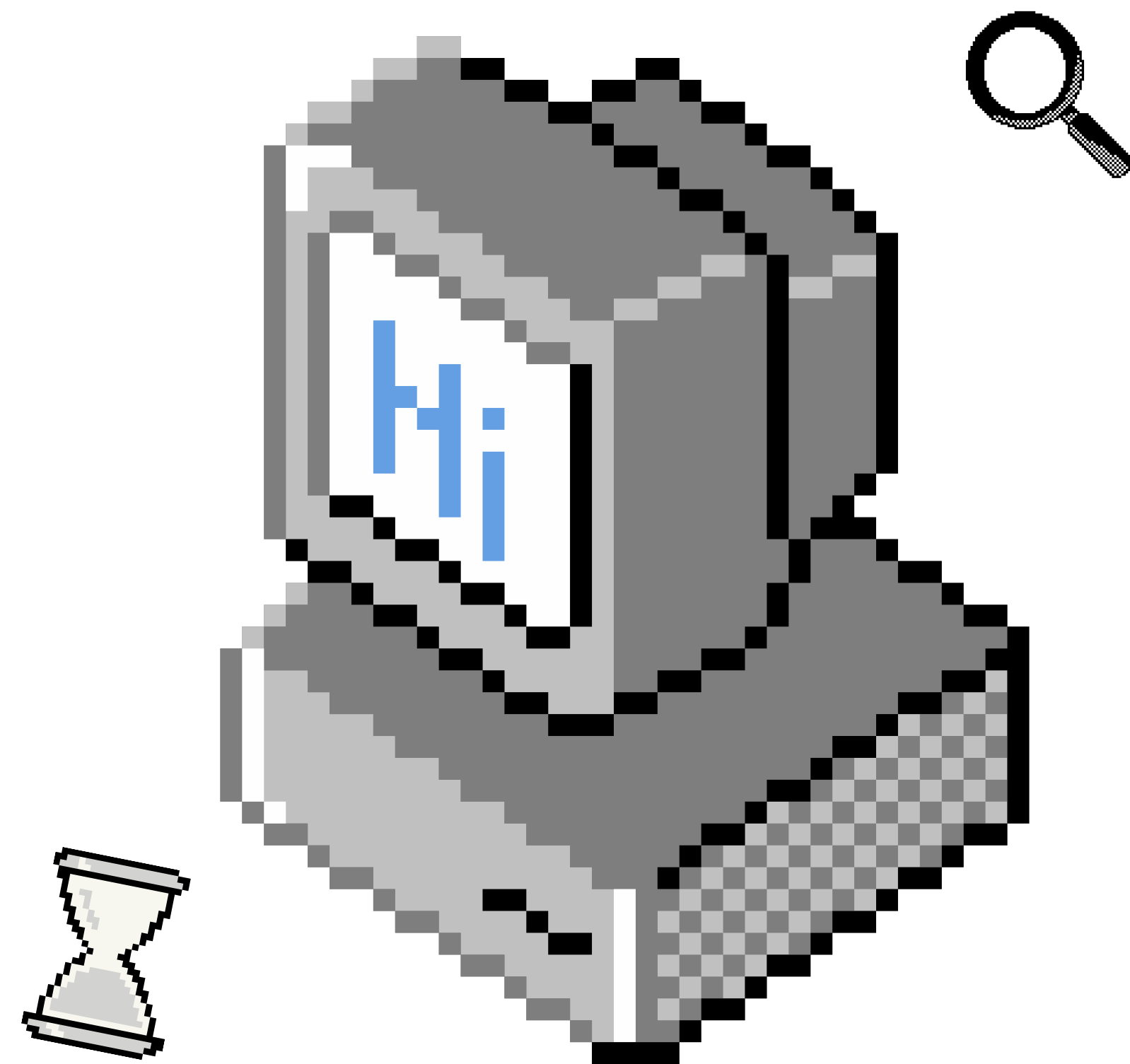




PRÁCTICA 6

Equipo 6

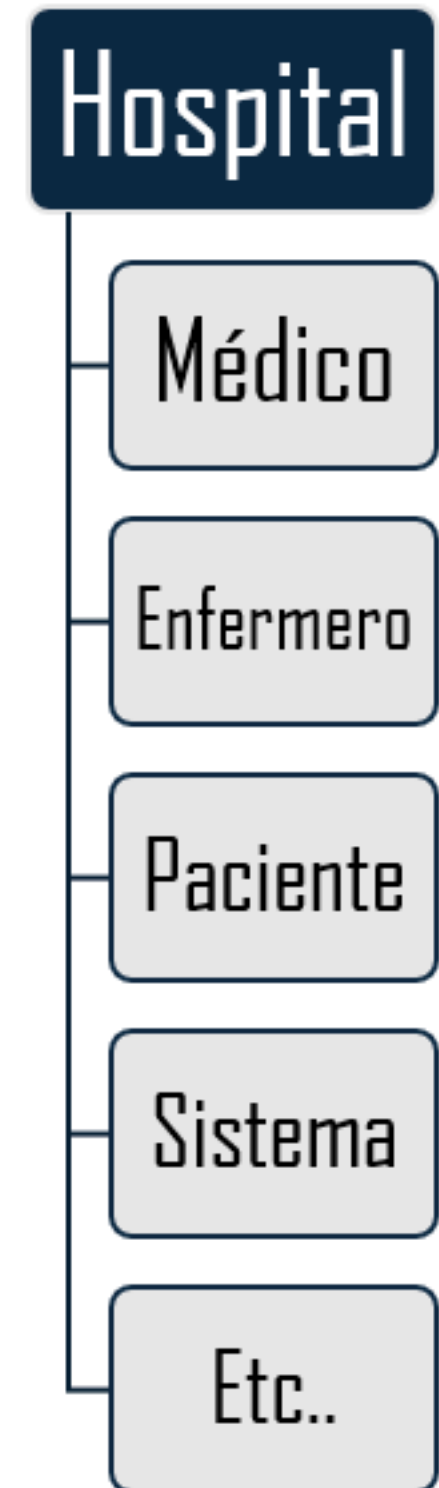
TEORÍA

Para comenzar a manejar muchos tipos de clases que se relacionan entre sí, se puede usar el concepto de composición, que en palabras simples es: “Un objeto se compone de otros objetos”.

Esto nos ayuda para:

- Agrupar clases relacionadas
- Evitar conflictos por clases que se llaman igual.

Como ejemplo podemos tomar el primer proyecto que realizamos, donde un hospital consistía de varias clases como médico, enfermero, paciente etc. Una forma de organizarlos de una manera en donde estén relacionados es en la figura de la derecha.



TEORÍA

Un archivo JAR (Java Archive) es un formato de archivo comprimido basado en ZIP que empaqueta código, bibliotecas y recursos de Java en una sola unidad para facilitar su distribución y ejecución.

Javadoc es una herramienta oficial de Oracle para generar documentación de interfaces de programación de aplicaciones (API) en formato HTML a partir del código fuente en Java.

A continuación expondremos de manera breve el manejo de estas estructuras de control, mediante la realización de las siguientes actividades:

ACTIVIDAD 1

1. Consultar con IA: buenas prácticas para uso de JAR y javadoc, además preguntar ¿cuál es la mejor manera de generar un javadoc?

Para jar obtuvimos el siguiente resultado:

6. Buenas prácticas para JAR

1. Usa nombres descriptivos

En lugar de:

```
programa.jar
```

es mejor:

```
sistema-bancario-1.0.jar
```

Y si utilizas versiones:

```
sistema-bancario-1.0.0.jar  
sistema-bancario-1.1.0.jar  
sistema-bancario-2.0.0.jar
```

Esto se vuelve especialmente importante en proyectos grandes.

2. Usa paquetes

No pongas todas las clases en el paquete por defecto.

Por ejemplo:

```
com.miempresa.banco
```

y:

```
</> Java  
package com.miempresa.banco;
```

Esto ayuda a organizar el código y evita conflictos entre clases.

3. No metas archivos innecesarios

Un JAR debería contener lo necesario para que la biblioteca o aplicación funcione.

Evita incluir cosas como:

```
*.java  
archivos temporales  
logs  
carpetas personales
```

si no son necesarios.



4. Usa control de versiones

Es una buena práctica que el nombre o sistema de distribución permita distinguir versiones:

```
biblioteca-1.0.0.jar  
biblioteca-1.1.0.jar  
biblioteca-2.0.0.jar
```

Más adelante aprenderás herramientas como **Maven** o **Gradle**, que automatizan gran parte de esto.

ACTIVIDAD 1

Para java doc obtuvimos el siguiente resultado:

7. Buenas prácticas para Javadoc

La regla más importante es:

Documenta lo que el programador necesita saber, no lo que el código hace.

Por ejemplo, esto aporta muy poco:

```
</> Java

/**
 * Suma dos números.
 */
public int sumar(int a, int b) {
    return a + b;
}
```

No está mal, pero en un método tan sencillo puede ser innecesario.

En cambio, si existe una condición especial:

```
</> Java

/**
 * Retira dinero de la cuenta.
 *
 * <p>La operación no puede dejar el saldo por debajo de cero.</p>
 *
 * @param cantidad cantidad que se desea retirar
 * @return true si el retiro fue realizado correctamente,
 *         false si no hay saldo suficiente
 */
public boolean retirar(double cantidad) {
    ...
}
```

La documentación aporta información que **no necesariamente se deduce inmediatamente del nombre del método.**

ACTIVIDAD 1

8. Documenta principalmente clases y métodos públicos

Por ejemplo:

```
</> Java

/**
 * Representa una cuenta bancaria.
 */
public class CuentaBancaria {

    /**
     * Deposita dinero en la cuenta.
     *
     * @param cantidad cantidad a depositar
     */
    public void depositar(double cantidad) {
        ...
    }
}
```

No necesitas llenar cada variable privada de comentarios.

9. Una buena práctica importante: mantener Javadoc actualizado

Supongamos que originalmente tienes:

```
</> Java

/**
 * @return el saldo de la cuenta
 */
public double getSaldo() {
    return saldo;
}
```

Después cambias el método para que devuelva otra cosa, pero no actualizas el comentario.

Ahora tienes:

```
Código ✗
Documentación ✗
```

Por eso:

La documentación debe evolucionar junto con el código.

Un Javadoc incorrecto puede ser incluso más perjudicial que no tener documentación.

ACTIVIDAD 1

Después le preguntamos cual era la mejor manera de generar un javadoc, y esto fue lo que obtuvimos:

Para ti, que estás aprendiendo Java y trabajas con VS Code, la mejor manera es **generar Javadoc desde la terminal usando la herramienta `javadoc` del JDK**. Es sencillo y te ayuda a entender qué está ocurriendo.

1. Primero escribe Javadoc en tus clases

2. Genera la documentación

Desde la terminal, estando en la carpeta donde está tu código:

```
</> Bash  
  
javadoc -d docs CuentaBancaria.java
```

`-d docs` significa:

"Guarda la documentación generada dentro de una carpeta llamada `docs`."

Obtendrás algo parecido a:

```
MiProyecto/  
├─ CuentaBancaria.java  
└─ docs/  
    ├─ index.html  
    ├─ CuentaBancaria.html  
    ├─ package-summary.html  
    └─ ...
```

3. Abre `index.html`

Puedes abrir:

```
docs/index.html
```

en tu navegador y tendrás una página web con la documentación de tu clase.

¿Y si tengo varias clases?

Aquí es donde conviene utilizar **paquetes**.

Por ejemplo:

```
src/  
└─ banco/  
    ├─ CuentaBancaria.java  
    ├─ Cliente.java  
    └─ Banco.java
```

Si las clases tienen:



ACTIVIDAD 2

En Java, los paquetes se usan para organizar los archivos y evitar que los nombres de los códigos choquen entre sí.

- mx: México.
- unam: Universidad Nacional Autónoma de México.
- fi: Facultad de Ingeniería.
- die: División de Ingeniería Eléctrica.
- poo: Programación Orientada a Objetos.
- g7: Grupo 7.
-

Una vez que llegamos al nivel de g7, la carpeta se divide en cuatro áreas de trabajo:

- src (Source): Todos los archivos (.java).
- lib (Libraries): Se usa para guardar archivos de código hechos por otras personas que el proyecto necesita para funcionar (como los archivos .jar externos).
- test (Pruebas): Se reserva para los códigos que sirven para probar y verificar que los códigos de la carpeta src no tengan errores.
- doc (Documentación): Aquí se guardan los reportes o la documentación generada automáticamente (como Javadoc) para que otros entiendan cómo funciona el código.

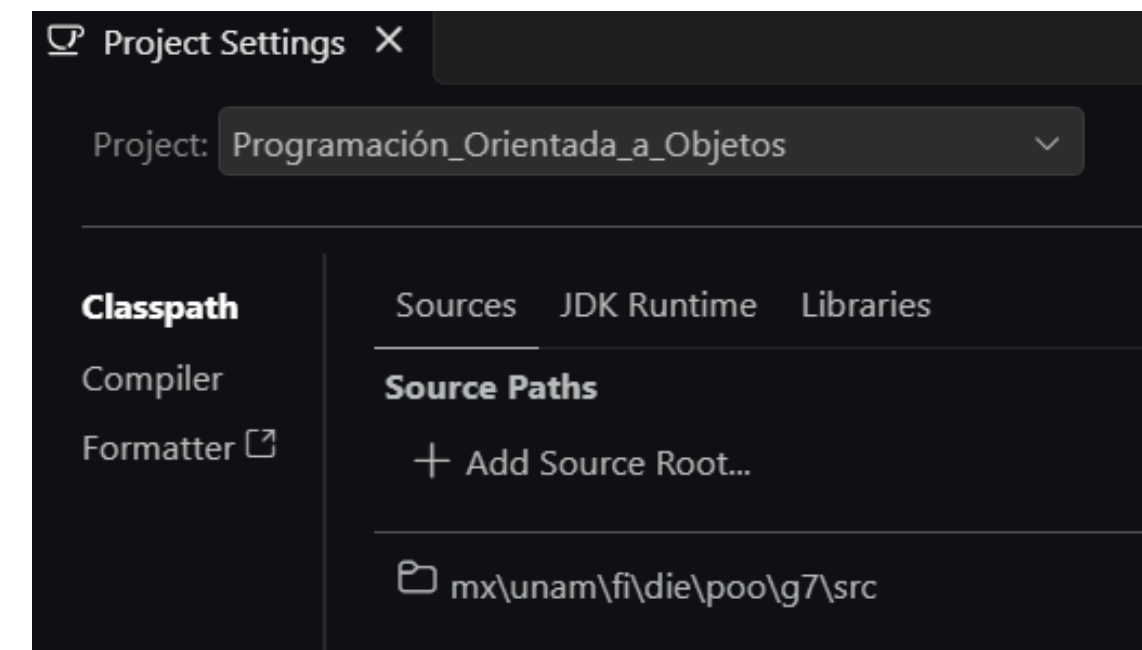
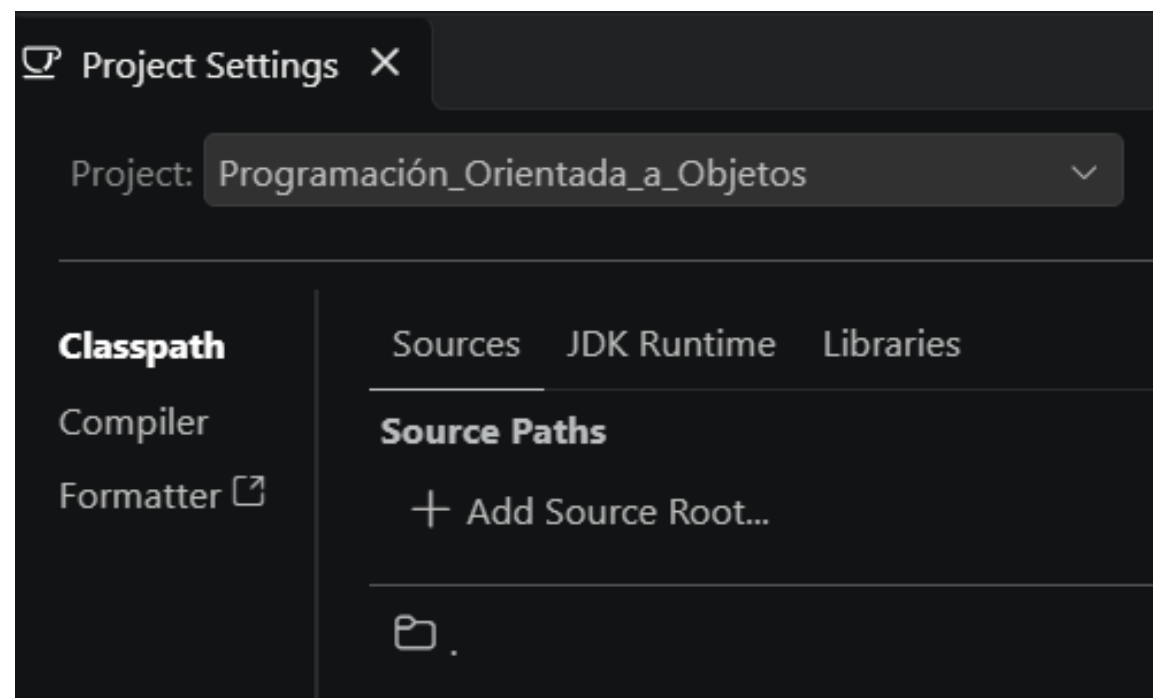


ACTIVIDAD 2

Una vez tenemos la estructura del proyecto, solo tenemos que importar los paquetes a los archivos.

Para esto podemos indicar la ruta completa `mx.unam.fi.die.poo.g7.src` por ejemplo.

Pero también podemos crear una `ruta raíz` (`Source Root`) para nuestro proyecto de java y solamente especificar la ruta final del paquete



```
// Importes
package mx.unam.fi.die.poo.g7.src.Práctica_1;
```

```
// Importes
package Práctica_1;
```